

南開大學
Nankai University

计算机学院

并行程序设计 **Lab1** 报告

CPU 架构编程

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 3 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

目录

一、实验一：$n*n$ 矩阵与向量内积 **2**
（一）算法设计 . . . 2
1. 平凡算法设计思路 . . . 2
2. 优化算法设计思路 . . . 2
（二）编程实现 . . . 2
1. 平凡算法 . . . 2
2. 优化算法 . . . 3
（三）性能测试 . . . 3
1. 性能测试设计 . . . 3
2. 性能测试结果 . . . 4
（四）profiling . . . 5
1. Hotspots 分析 . . . 5
2. HEBS 补充分析 . . . 5

二、实验二：n 个数求和 **5**
（一）算法设计 . . . 6
1. 平凡算法设计思路 . . . 6
2. 优化算法设计思路 . . . 6
（二）编程实现 . . . 6
1. 平凡算法 . . . 6
2. 优化算法 . . . 6
（三）性能测试 . . . 6
1. 性能测试设计 . . . 6
2. 性能测试结果 . . . 7
（四）profiling . . . 8
1. Hotspots 分析 . . . 8
2. Microarchitecture Exploration 分析 . . . 8

三、实验总结和思考 **8**
（一）对比 2 个实验的异同 . . . 8
（二）总结 . . . 8

一、实验一：$n * n$ 矩阵与向量内积

实验目的

本实验围绕 $n \times n$ 矩阵与向量内积的计算展开，实现并比较两种算法：逐列访问矩阵元素的平凡算法，以及按行访问矩阵、利用缓存空间局部性的 cache 优化算法。实验目标是通过编程实现、高精度计时与 profiling，分析不同访存模式对程序性能的影响，并理解 cache 层次结构在实际程序运行中的作用。

实验环境

两组实验均在本地单机环境下完成。实验平台为 Microsoft Windows 11，处理器为 13th Gen Intel(R) Core(TM) i5-13500H，12 个物理核心、16 个逻辑处理器，基准主频 2.60 GHz，内存 32 GB。实验一额外关注 cache 层次结构，处理器缓存配置为：L1 Cache 1.1 MB，L2 Cache 9.0 MB，L3 Cache 18.0 MB。程序使用 `g++ 13.3.0` 编译，编译选项为 `-O2 -g -std=c++17`。其中 `-O2` 用于启用优化，`-g` 用于配合 VTune 进行 profiling。

问题描述

给定矩阵 $B \in \mathbb{R}^{n \times n}$ 和向量 $a \in \mathbb{R}^n$，计算矩阵每一列与向量 a 的内积，得到结果向量 sum，即

$$sum[i] = \sum_{j=0}^{n-1} B[j][i] \cdot a[j], \quad i = 0, 1, \ldots, n-1. \tag{1}$$

从运算次数来看，两种算法都需要进行 $O(n^2)$ 次乘加操作，因此它们的渐近时间复杂度相同。实验中需要重点考察的不是计算量差异，而是不同访存模式所导致的 cache 行为差异。

测试数据生成

为保证实验可复现并便于正确性验证，本实验不使用随机输入，而是采用确定性规则生成测试数据。我们定义

$$\text{deterministic_value}(i, j) = \frac{(i + j) \bmod 1024}{17.0} - 20.0, \tag{2}$$

矩阵元素设置为 $B[r][c] = \text{deterministic_value}(r, c)$，向量元素设置为 $a[i] = \text{deterministic_value}(i, i + 3)$。这种固定生成方式能够避免随机输入带来的额外波动，使性能比较更集中地反映访存模式差异。

（一）算法设计

1. 平凡算法设计思路

平凡算法按定义直接计算：外层按列、内层按行，累加 $B[j][i] \cdot a[j]$ 得到各列内积。该方法实现简单，但逐列访问与矩阵行主序存储不匹配，访存不连续，cache 局部性较差。

2. 优化算法设计思路

优化算法改为按行访问：先清零结果数组，再逐行读取矩阵，将 $B[j][i] \cdot a[j]$ 累加到 `sum[i]` 中。该方法虽不能在一次外层循环中直接得到单列结果，但访存更连续，空间局部性更好，因此性能通常优于平凡算法。

（二）编程实现

1. 平凡算法

```
for (i = 0; i < n; i++) {
    sum[i] = 0.0;
    for (j = 0; j < n; j++) {
        sum[i] += b[j][i] * a[j];
    }
}
```

2. 优化算法

```
for (i = 0; i < n; i++) {
    sum[i] = 0.0;
}
for (j = 0; j < n; j++) {
    for (i = 0; i < n; i++) {
        sum[i] += b[j][i] * a[j];
    }
}
```

两种实现的区别仅在于循环次序。平凡算法按列访问矩阵，优化算法按行访问矩阵，因此它们的性能差异可以较直接地归因于访存模式的不同。

（三）性能测试

1. 性能测试设计

本实验使用 C++ `std::chrono::steady_clock` 对核心内积函数计时，不将测试数据生成、正确性校验和 CSV 输出等辅助操作计入结果。对于单次执行时间较短的小规模问题，程序采用自适应重复测试策略：若累计时间不足约 220 ms，则逐步增加 `repeats`；随后进行 5 轮独立测量，并取最小单次运行时间作为该规模下的结果。为避免编译器优化掉核心计算，程序将部分结果累加到 `volatile` 全局变量中。

测试规模取

$$n = 64, 128, 192, 256, 320, 384, 448, 512, 640, 768, 896, 1024, 1280, 1536, 1792, 2048,$$

共 16 个测试点，对应工作集大小约从 0.032 MiB 增长到 32.031 MiB，覆盖从明显小于 cache 到接近或超过各级 cache 容量的多个区间。测试数据采用确定性规则生成，以保证实验可复现并便于正确性验证。

工作集主要由矩阵 B、向量 a 和结果向量 `sum` 构成，其大小近似为

$$W_{\mathrm{MiB}} = \frac{(n^2 + 2n) \times 8}{1024^2}.$$

以工作集大小为横轴更便于结合 cache 层次分析性能变化。

2. 性能测试结果

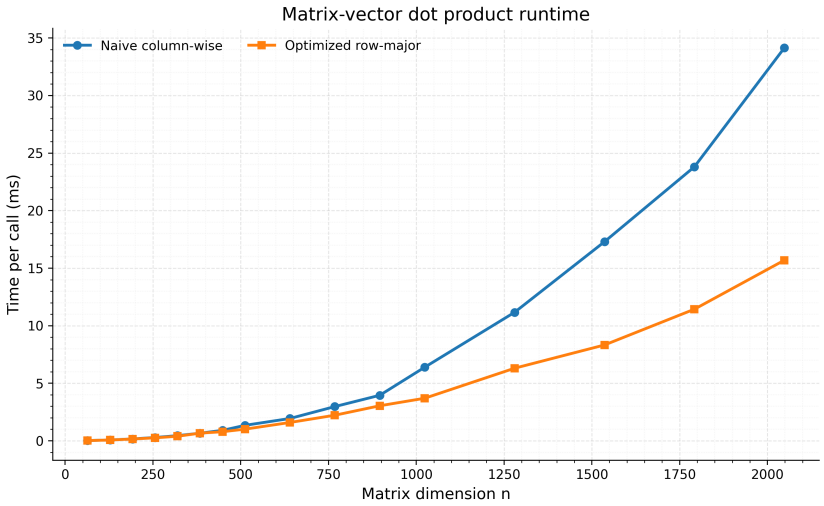


图 1: 两种算法单次运行时间随矩阵维度 n 的变化

图 1 表明，随着矩阵规模增大，两种算法运行时间均上升，但平凡算法增长更快，优化算法增长相对平缓。这说明在大规模情况下，访存开销逐渐成为主要瓶颈，而按行访问的优化算法由于更符合行主序存储方式，能够获得更好的缓存性能。

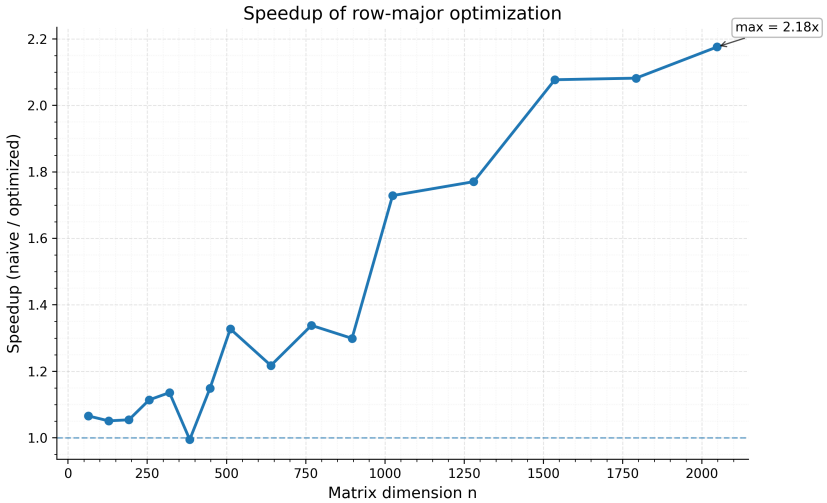


图 2: 逐行访问优化算法相对于平凡算法的加速比

图 2 给出了优化算法相对于平凡算法的加速比

$$\text{Speedup} = \frac{T_{\text{naive}}}{T_{\text{optimized}}}.$$

可以看到，加速比总体随矩阵规模增大而上升：小规模时接近 1，而大规模时超过 2，最大约为 2.18×。虽然中间存在少量波动，但整体趋势清楚地表明，逐行访问在大规模问题上具有显著优势。

（四）profiling

1. Hotspots 分析

VTune Hotspots 结果表明，程序开销几乎全部集中在两个核心函数中：`dot_row_major` 占 51.0%，`dot_naive_col` 占 48.7%。这说明 benchmark 的主要耗时确实来自待研究的矩阵内积计算，而非初始化、输出等外围代码。

同时，程序总线程数为 1，说明实验在单线程条件下完成。需要说明的是，Hotspots 结果用于定位瓶颈函数，不能直接替代固定规模下的精确性能比较；算法优劣仍应以前述 benchmark 结果为准。

2. HEBS 补充分析

为进一步解释两种算法的性能差异，本实验使用 VTune 的 Hardware Event-Based Sampling (HEBS) 采集 cache hit/miss 事件。结果如表 1 所示。

表 1: 两种算法的 cache 命中率

Function	L1 Rate	L2 Rate	L3 Rate
`dot_row_major`	99.5664%	97.8340%	49.2701%
`dot_naive_col`	94.1929%	52.3619%	92.1354%

从表中可以看出，`dot_row_major` 在 L1/L2 cache 上的 hit 明显更多，说明其大部分访存可以在较高层 cache 中完成；而 `dot_naive_col` 在 L1/L2 上出现了更多 miss，导致更多访问继续穿透到更低层 cache，访存代价更高。

需要注意的是，`dot_naive_col` 的 L3 命中率虽然更高，但这并不意味着其 cache 表现更优；相反，这通常说明它由于 L1/L2 命中不足，有更多访问落入 L3。

这一结果与 benchmark 和理论分析一致：逐列访问与矩阵的行主序存储不匹配，空间局部性较差；逐行访问更符合存储布局，因此 cache 利用更充分，在大规模问题下表现出更优性能。

二、实验二：n 个数求和

实验目的

本实验围绕 n 个数求和问题，比较链式平凡算法与适合超标量架构的指令级并行算法。通过编程实现、高精度计时和 VTune profiling，分析指令依赖对性能的影响，并理解多链路展开与归约策略的加速机理。

实验环境

同实验一。

问题描述

给定长度为 n 的数组 a，计算其元素总和

$$S = \sum_{i=0}^{n-1} a_i.$$

虽然不同实现的时间复杂度同为 $O(n)$，但循环展开方式、独立累加链数量和归约策略不同，会导致指令级并行能力存在差异。

测试数据生成

为保证实验可复现，本实验采用确定性规则生成测试数据，不使用随机输入。数组长度取 2 的幂，便于实现两路展开、四路展开和 pairwise 归约算法。

（一）算法设计

1. 平凡算法设计思路

平凡算法使用单个累加变量顺序遍历数组并依次求和。该方法实现简单，但存在严格的数据依赖，不利于发挥处理器的指令级并行能力。

2. 优化算法设计思路

为减弱串行依赖，本实验采用两路展开、四路展开和 pairwise 归约三种优化方法。前两者通过增加独立累加链提升并行性，后者通过分层归约缩短依赖链，从而更适合 superscalar 处理器执行

（二）编程实现

1. 平凡算法

```cpp
double sum = 0.0;
for (std::size_t i = 0; i < n; ++i) {
    sum += a[i];
}
```

2. 优化算法

两路展开

```cpp
double sum1 = 0.0, sum2 = 0.0;
for (std::size_t i = 0; i < n; i += 2) {
    sum1 += a[i];
    sum2 += a[i + 1];
}
double sum = sum1 + sum2;
```

四路展开

```cpp
double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;
for (std::size_t i = 0; i < n; i += 4) {
    s0 += a[i];
    s1 += a[i + 1];
    s2 += a[i + 2];
    s3 += a[i + 3];
}
double sum = (s0 + s1) + (s2 + s3);
```

pairwise 迭代归约的思想为：在长度为 m 的当前数组段上，将相邻元素两两相加并连续写回数组前部，然后令 $m \leftarrow m/2$，不断重复，直到剩余一个元素为止。

（三）性能测试

1. 性能测试设计

本实验采用高精度计时方法测量核心求和函数时间，不将数据生成、结果输出等辅助操作计入性能结果。对于执行时间较短的小规模问题，程序通过重复执行待测函数并进行多轮独立测试，以减小计时误差。

数组长度取 2 的幂，测试规模为

$$n = 1024,\ 4096,\ 16384,\ 65536,\ 262144,\ 1048576,\ 4194304,\ 16777216.$$

对应数据规模约从 0.01 MiB 增长到 128 MiB。这些规模覆盖了从较小到较大的不同工作区间，便于比较各算法在不同问题规模下的性能表现。

2. 性能测试结果

与链式算法相比，两路展开、四路展开和 pairwise 迭代归约均取得了加速，其中四路展开整体表现最好，pairwise 归约次之，两路展开也具有稳定收益。

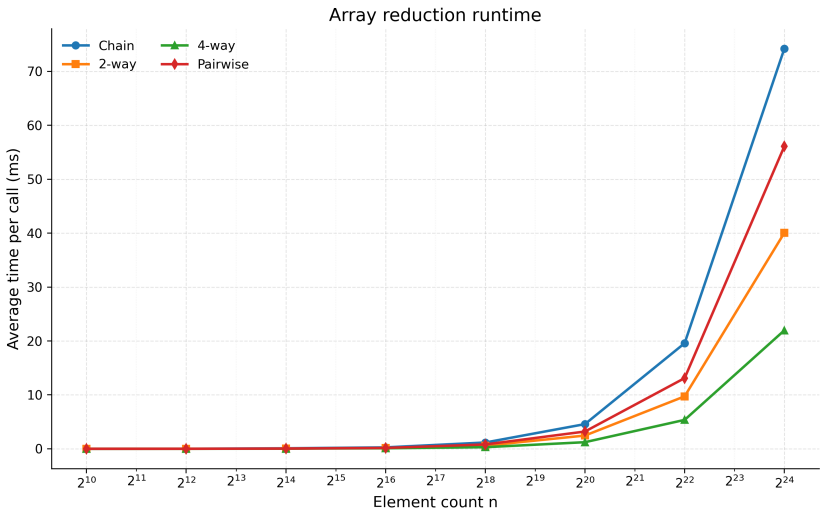


图 3: 不同求和算法单次运行时间随数组长度 n 的变化

图 3 表明，随着数组长度增大，四种算法运行时间均上升，但链式算法始终最慢；两路展开和四路展开明显更优，其中四路展开整体最快；pairwise 归约虽优于链式算法，但整体略逊于四路展开。

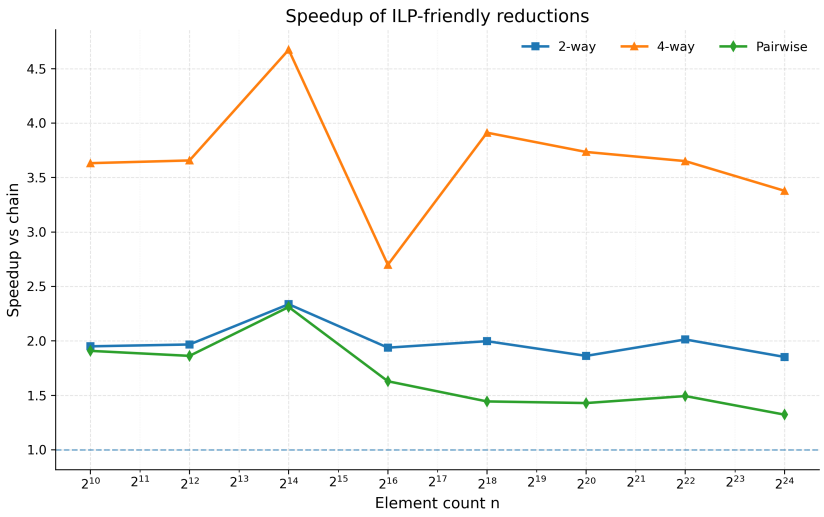


图 4: 优化算法相对于链式算法的加速比

图 4 给出了三种优化算法相对于链式算法的加速比。可以看到，四路展开的加速最为显著，两路

展开也具有稳定收益，而 pairwise 归约虽能带来正加速，但整体不及四路展开。这说明增加独立累加链、削弱指令依赖能够有效提升程序性能。

（四）profiling

Profiling 方法说明 实验二的 profiling 分为两步：首先利用 VTune Hotspots 定位主要耗时函数，确认 benchmark 的 CPU 开销主要集中在核心求和函数；随后利用 Microarchitecture Exploration 分析执行周期数、退役指令数和 CPI，从微结构层面解释各算法的性能差异。

1. Hotspots 分析

VTune Hotspots 结果表明，除测试辅助函数 `flush_cache` 外，程序的主要 CPU 开销集中在 `sum_chain`、`sum_two_way`、`sum_four_way_unroll` 和 `sum_pairwise_iterative` 等核心求和函数中，说明 benchmark 的主要耗时确实来自待研究的求和计算部分。

2. Microarchitecture Exploration 分析

为进一步解释性能差异，本实验使用 VTune 的 Microarchitecture Exploration，重点考察执行周期数、退役指令数和 CPI。结果如表 2 所示。

表 2: 实验二各核心函数的 Microarchitecture Exploration 结果

Function	CPU Time (s)	Clockticks	Instructions Retired	CPI
`sum_chain`	8.075	15193600000	22582400000	0.673
`sum_two_way`	8.185	15523200000	41382400000	0.375
`sum_four_way_unroll`	8.964	16972800000	75254400000	0.226
`sum_pairwise_iterative`	7.896	14752000000	73094400000	0.202

从表 2 可以看出，`sum_chain` 的 CPI 最高，说明链式算法中的连续加法具有较强数据依赖，限制了指令级并行能力。相比之下，两路展开、四路展开和 pairwise 归约都显著降低了 CPI，表明这些方法能够通过增加独立累加链或缩短依赖链，更充分地发挥 superscalar 处理器的执行能力。

总体来看，profiling 结果与 benchmark 结果一致：链式算法依赖链最长，性能最差；而两路展开、四路展开和 pairwise 归约均能削弱串行依赖，因此获得更好的性能。

三、实验总结和思考

（一）对比 2 个实验的异同

两个实验都表明：即使算法的渐近复杂度相同，程序的实际性能仍会因底层硬件特性而产生明显差异。实验一的性能差异主要来自访存顺序不同所造成的 cache 局部性差异；实验二的性能差异主要来自指令依赖强弱不同所带来的指令级并行能力差异。

两组实验的共同点在于都没有改变问题本身，而是通过更好地匹配硬件执行机制来提升性能。不同之处在于，实验一侧重存储层次优化，实验二侧重处理器执行层次优化。

（二）总结

通过本次 Lab1，可以看出程序性能不仅取决于算法复杂度，还与数据布局、访存模式、循环结构和指令依赖密切相关。因此，在体系结构相关程序设计中，应重视算法实现方式与底层硬件特性的匹配，以获得更好的实际执行效率。